

Software Engineering Interview Study Guide

SDLC • Agile • Scrum

Nine topic areas, discovered from current interview-report and practitioner sources rather than a fixed syllabus — spanning SDLC models, the Agile Manifesto, the official Scrum Guide, Kanban and scaled frameworks, estimation and delivery metrics, requirements engineering, testing strategy, CI/CD, and the anti-patterns that actually sink teams.

72 questions • Basic → Intermediate → Advanced

9 topic areas

Templates: user story • Gherkin AC • DoR/DoD • CI/CD stages

Sourced from Scrum Guide, Atlassian, GfG, Glassdoor, Scrum.org, DORA

Updated Sep 2026

WHO THIS IS FOR

Intern

Junior

Mid

Senior

- **Intern/Junior** — expect direct definitions and comparisons (Sections 1–3, 6–7).
- **Mid-level** — expect scenario and anti-pattern questions probing how you'd actually handle a sprint under pressure.
- **Senior/Lead/Scrum Master** — expect trade-off framing across whole sections (5, 8, 9): metrics you'd distrust, deployment risk calls, and why a team's process is quietly failing.

WHAT INTERVIEWERS EVALUATE

- **Correct terminology** — using "Sprint Goal," "Increment," "Definition of Done" precisely, not interchangeably.
- **Trade-off reasoning** — not "Agile is better than Waterfall," but "here's when each earns its cost."
- **Real examples** — a specific sprint, ticket, incident, or release, not textbook abstractions.
- **Judgment under ambiguity** — scenario questions have no single correct answer; interviewers grade the reasoning path.

01 SDLC Fundamentals & Models

Before Agile, before Scrum, there is the Software Development Life Cycle itself — the phase model every methodology is an answer to. Interviewers use this section to confirm you understand *why* Agile exists, not just how to run a standup.

CORE DEFINITIONS

SDLC (Software Development Life Cycle)

The structured sequence of phases — typically requirements, design, implementation, testing, deployment, and maintenance — that a software product moves through from conception to retirement.

Plan-driven vs iterative development

Plan-driven models (Waterfall, V-Model) fix scope and sequence upfront; iterative models (Agile, Spiral) re-plan repeatedly as the team learns.

1.1 **BASIC** CONCEPT What is the SDLC, and what are its canonical phases?

The Software Development Life Cycle is the structured process a team follows to plan, build, test, and maintain software. Nearly every methodology — however much it reorders, overlaps, or repeats these phases — is built from the same underlying activities: **requirements gathering & analysis** (what should the system do), **design** (architecture and detailed design of how it will do it), **implementation/coding, testing** (verifying it works and meets requirements), **deployment/release** (delivering it to users), and **maintenance** (fixing defects, adapting to change, once it's live). The academic framing is a life cycle because maintenance routinely feeds back into new requirements — the "cycle" doesn't end at release.

requirements analysis

maintenance

life cycle

1.2 **BASIC** CONCEPT

Explain the Waterfall model — its structure, strengths, weaknesses, and when it's still the right call.

Waterfall is a **linear, sequential** model: each phase must fully complete before the next begins, and phases are not meant to overlap or revisit earlier decisions. Its strength is predictability — with fixed requirements, a fixed scope, a fixed schedule and budget are genuinely achievable, and the heavy upfront documentation makes it auditable, which matters for regulated or contractually fixed-price work. Its weakness is the flip side of the same rigidity: because testing happens only after implementation is "complete," a requirements misunderstanding discovered in acceptance testing can mean redoing large amounts of design and code, at the most expensive point in the project to fix it. Waterfall remains the right call for projects with genuinely stable, well-understood requirements and low tolerance for scope drift — certain government, aerospace, and compliance-driven contracts — but is a poor fit whenever the problem itself is still being discovered.

linear/sequential

upfront documentation

late defect discovery

1.3

BASIC

COMPARISON

Explain the V-Model, and how it differs from Waterfall.

The V-Model extends Waterfall by pairing every development phase with a corresponding testing phase, drawn as a V-shape: as you descend the left side (requirements → system design → architecture → module design), you ascend the mirrored right side (unit testing → integration testing → system testing → acceptance testing), with each test phase explicitly validating the design decision made at its mirrored point. The core difference from plain Waterfall is that **test planning happens in parallel with each design phase**, not as an afterthought once coding finishes — acceptance test criteria are written against the original requirements the moment those requirements are captured, not months later.

	WATERFALL	V-MODEL
Testing timing	After implementation, as one late phase	Planned alongside each corresponding design phase
Defect discovery	Often late, expensive to fix	Earlier, because test cases exist before code does
Best for	Stable, well-documented requirements	Verification-heavy domains (medical devices, avionics)

V-shape mirroring

verification alongside design

1.4

INTERMEDIATE

CONCEPT

Explain the Spiral model, and why it's called a "meta-model."

The Spiral model, proposed by Barry Boehm, is called a meta-model because it doesn't prescribe one fixed sequence of phases — it wraps Waterfall-like, iterative, and prototyping approaches inside a repeating cycle of four activities per "loop" around the spiral: **determine objectives** for this iteration, **identify and analyze risk** (its signature addition — every loop starts with explicit risk assessment), **develop and test** the next increment, and **plan the next iteration**. Each pass around the spiral produces a more complete, more validated version of the system, with risk analysis explicitly gating whether the project continues, pivots, or is killed. It's the right choice for large, high-uncertainty, high-stakes projects (novel technology, large budgets, unclear requirements) where the cost of formal risk analysis each iteration is justified by what a late-discovered fatal risk would otherwise cost.

Boehm

risk-driven iteration

meta-model

Incremental vs Iterative vs Agile development — what's the actual distinction?

MODEL	CORE IDEA	WHAT CHANGES BETWEEN PASSES
Incremental	Build and deliver the system in separate functional pieces	New features are added; earlier pieces are largely untouched
Iterative	Build a rough version of the whole system, then refine it repeatedly	The same functionality is revisited and improved each pass
Agile	Iterative + incremental, plus short feedback loops with real users/stakeholders and continuous re-prioritization	Both scope and priority can change every cycle based on feedback

In practice, Agile is best understood as iterative-incremental development with a specific value system layered on top (the Agile Manifesto, covered next) — the mechanics of "build a slice, get feedback, adjust" predate the term "Agile" by decades; what Agile added was the explicit prioritization of responding to change and customer collaboration over rigidly following a plan.

incremental

iterative

feedback loop

What is RAD (Rapid Application Development), and when is it appropriate?

RAD prioritizes speed and working prototypes over rigorous upfront planning: it compresses the traditional design phase into rapid, user-involved prototyping cycles, often leaning on reusable components and low-code/no-code tooling to shrink build time. It works well when requirements are business-driven and relatively well-understood by users even if not formally documented, the user base is available for frequent feedback, and the system doesn't carry extreme performance or safety constraints (internal business tools, dashboards, CRUD-heavy line-of-business apps). It's a poor fit for large, technically complex, or highly regulated systems where prototyping speed can't substitute for architectural rigor.

prototyping

reusable components

You're building avionics software with fixed regulatory requirements, and separately, a startup's first customer-facing MVP with unknown market fit. Which SDLC model for each, and why?

For the avionics system: a **V-Model** (or Waterfall with heavy formal verification) is the defensible choice. Requirements come from regulation (e.g. DO-178C for avionics software) and are not going to change based on user feedback; the cost of a defect reaching production is measured in safety, not just money; and the paired verification-at-every-phase structure produces the audit trail regulators require. Iterating and "learning" the requirements through shipped increments is not an option when the requirements are externally fixed and the failure mode is catastrophic.

For the startup MVP: **Agile**, run in very short cycles, is the defensible choice, precisely because the core unknown isn't "can we build this correctly" but "should we build this at all" — the fastest, cheapest way to learn whether real users want the product is to ship a thin, working slice and observe actual usage, then re-prioritize the backlog based on what's learned. Using Waterfall here would mean spending months building against requirements nobody has validated yet.

The interview-grade point: SDLC choice is not an ideological preference, it's a response to **where the uncertainty lives** — in the requirements (favor iteration) or in the execution against known requirements (favor plan-driven rigor).

DO-178C uncertainty-driven model choice

What is the Cone of Uncertainty, and how does it justify different SDLC choices at different project stages?

The Cone of Uncertainty (from Steve McConnell's estimation research) describes how estimate accuracy narrows over a project's life: at the very start (initial concept), an estimate can be off by a wide margin in either direction; by the time detailed requirements exist, the range narrows considerably; by the time the system is nearly complete, remaining uncertainty is small. The cone shape is the argument *against* committing to a single fixed estimate, schedule, and scope at project kickoff — the point of maximum ignorance — and the argument *for* iterative models: each completed iteration is a source of real information that narrows the cone faster than upfront analysis alone can, which is exactly why Agile teams re-estimate and re-forecast every sprint rather than trusting a Day-1 estimate for the life of the project.

Cone of Uncertainty McConnell progressive elaboration

WHAT THE WEB SOURCES EMPHASIZE

GeeksforGeeks and InterviewBit's SDLC question sets consistently center on "compare Model X vs Model Y, and say when you'd use each" rather than pure definitions — the searched sources reward trade-off framing (schedule/budget certainty vs adaptability) over rote recall of phase names.

02 Agile Manifesto & Agile vs Traditional

Glassdoor-reported interviews return to a small cluster of questions again and again here: Agile vs Waterfall, Agile vs Scrum, MVP, Sprint o, and "spike." This section covers exactly that cluster, plus the values underneath it.

2.1 BASIC CONCEPT

State the Agile Manifesto's four values — what do they mean in practice, not just as slogans?

The 2001 Agile Manifesto states four value pairs, each phrased as "we value the item on the left more, while still valuing the item on the right":

1. **Individuals and interactions** over processes and tools — a well-functioning team with a mediocre tracking tool outperforms a poorly-functioning team with a perfect one.
2. **Working software** over comprehensive documentation — a demoable increment is stronger proof of progress than a status document describing planned progress.
3. **Customer collaboration** over contract negotiation — ongoing dialogue catches misunderstandings early; a rigid contract only catches them at delivery, when it's expensive to fix.
4. **Responding to change** over following a plan — a plan is a starting hypothesis, not a commitment that overrides new information.

The consistently missed nuance, and the one interviewers listen for: the manifesto explicitly does *not* say the right-hand items have no value — tools, documentation, contracts, and plans still matter, just less than the item they're paired against when the two are in tension.

Agile Manifesto

value pairs, not absolutes

2.2 BASIC COMPARISON Agile vs Waterfall — full comparison.

	WATERFALL	AGILE
Planning	Comprehensive, upfront	Continuous, rolling, re-prioritized each cycle
Change tolerance	Low — changes are formally controlled	High — change is expected and designed for
Delivery cadence	One delivery, at the end	Frequent small increments (e.g. every sprint)
Customer involvement	Mainly at requirements and acceptance	Continuous, throughout
Documentation	Extensive, formal	"Just enough," favoring working software
Risk visibility	Often late (at integration/acceptance)	Early — each increment surfaces reality quickly

change tolerance

delivery cadence

2.3 BASIC CONCEPT

What is an MVP (Minimum Viable Product), and how does it differ from a prototype?

An MVP is the smallest version of a product that can be released to real users and still deliver enough value to generate genuine, actionable feedback — the point is learning from real usage, not showcasing an idea. A **prototype**, by contrast, is typically not production-grade and not meant for real users at all — it's a throwaway or semi-throwaway artifact (a clickable mockup, a proof-of-concept) used internally or with a small test group to validate a design or technical approach before committing engineering effort. The distinction interviewers probe for: an MVP must be genuinely usable and shippable, even if minimal — "minimum" modifies the feature set, not the quality bar.

MVP

prototype

learn from real usage

2.4 INTERMEDIATE CONCEPT What is "Sprint 0," and what's a "spike"?

These are two of the most commonly reported Glassdoor interview questions for Agile/Scrum roles, and they're often confused with each other. **Sprint 0** is an informal, not-officially-in-the-Scrum-Guide term for a short setup period before real sprint work begins — establishing the team's tooling, environment, initial backlog, and Definition of Done. It's controversial precisely because it isn't a real Scrum event and can become an excuse to front-load planning the framework is designed to avoid. A **spike** is a time-boxed investigation — a research or technical-exploration task (e.g. "can we integrate with this third-party API at all?") whose output isn't working software but knowledge that reduces uncertainty for a future story's estimate. Spikes can happen inside any sprint, not just at the start.

Sprint 0 spike time-boxed investigation

REAL INTERVIEW SIGNAL

Glassdoor-reported Agile interviews specifically flag "define DoD/DoR," "Sprint 0 vs spike," and "Agile vs Scrum" as recurring direct-recall questions — they're cheap to prepare and commonly used as an early filter.

2.5 INTERMEDIATE COMPARISON

Agile vs Scrum — what's the actual relationship between the two terms?

This is one of the most frequently reported real interview questions in the search results, and the correct answer is a category relationship, not a synonym pair. **Agile** is a set of *values and principles* (the Manifesto) — a philosophy about how to approach software delivery under uncertainty. **Scrum** is one specific, concrete *framework* that implements those values — with defined accountabilities, events, and artifacts. Other frameworks (Kanban, XP, Scrumban, SAgile) also implement Agile values, just with different mechanics. Saying "we do Agile" is like saying "we play sports" — Scrum is one specific sport (with its own named positions and rules), not the whole category.

philosophy vs framework

2.6 INTERMEDIATE CONCEPT

What does "self-organizing team" mean — and why is "no management" the wrong reading of it?

A self-organizing (now more precisely "self-managing" in the current Scrum Guide) team decides *internally* who does which work, and how, rather than having an outside manager assign tasks to individuals. It does not mean the team has no goals, no accountability, or no external stakeholders — the Product Owner still sets *what* and *why* (priority, value), the organization still sets constraints (budget, deadlines, compliance), and the Scrum Master still safeguards the process. What moves inside the team is the *how*: task breakdown, technical approach, and who picks up what. Interviewers who ask this are checking whether a candidate conflates "self-organizing" with "unmanaged," a misunderstanding that in practice produces exactly the chaos critics of Agile point to.

self-managing team

what vs how

2.7 ADVANCED SCENARIO

Leadership wants "Agile," but also wants a fixed scope, fixed date, and fixed budget committed upfront. How do you respond?

This is the classic **Iron Triangle** conflict: scope, schedule, and cost cannot all be fixed simultaneously when the fourth variable — quality, or more precisely, the unknowns still being discovered — is allowed to vary, and Agile's entire value proposition rests on treating scope as the flexible variable while holding schedule and cost roughly fixed (fixed-length sprints, roughly fixed team size). The professional response isn't "no" — it's reframing the conversation: agree on a fixed date and fixed team cost, and treat the *backlog priority order* as the lever, committing to delivering the highest-value items first and explicitly naming which lower-priority items may not make the cut, rather than promising a fixed list of features by a fixed date at a fixed cost, which is not something any methodology can honestly guarantee once real unknowns exist. Making this trade-off explicit, early, and in writing (rather than silently overcommitting) is the actual skill being tested here.

Iron Triangle

scope as the flexible variable

What is "Water-Scrum-Fall," and why does it happen in large enterprises adopting Agile?

Water-Scrum-Fall describes an organization that runs Scrum ceremonies for the development team in the middle of the process, while the front end (requirements gathering, budgeting, sign-off) and back end (release approval, compliance, deployment windows) remain rigidly Waterfall. The team holds standups and sprint reviews, but can't actually ship an increment at the end of a sprint because release still requires a quarterly change-approval board — so the "working software" the sprint produced sits in a queue, and the fast-feedback loop Agile depends on never closes. It happens because adopting Scrum ceremonies is organizationally cheap (one team can start doing standups tomorrow), while changing the surrounding budgeting, governance, and release-approval structures requires cross-organizational change that's far slower and more politically difficult — so companies adopt the visible mechanics of Agile without the structural changes that make those mechanics actually deliver value.

Water-Scrum-Fall

ceremony without structural change

WHAT THE WEB SOURCES EMPHASIZE

Glassdoor's aggregated Agile/Scrum Master interview reports repeatedly name the same small cluster — DoR/DoD, Sprint 0 vs spike, Agile vs Waterfall, Agile vs Scrum, MVP — as direct-recall screening questions, distinct from the scenario-heavy questions reserved for later rounds.

03 Scrum: Accountabilities, Events & Artifacts

Grounded directly in the official 2020 Scrum Guide. This is the highest-density interview section for Scrum Master and Agile Product Owner roles — expect both direct-recall and multi-step scenario questions here.

3.1 BASIC CONCEPT

Define Scrum per the official Scrum Guide — what problem is it actually solving?

Scrum is a lightweight framework for developing, delivering, and sustaining complex products, built around small teams working in short, fixed-length iterations called Sprints. It exists to solve the problem of **complex, adaptive work** — work where the requirements, technology, and market are not fully knowable upfront — by structuring frequent inspection and adaptation into the process itself rather than trusting a single upfront plan. It is explicitly a *framework*, not a complete methodology: it defines the minimal set of roles, events, and artifacts needed to enable empiricism, and deliberately leaves the specific engineering practices (how you test, how you design) to the team.

empiricism

complex adaptive work

framework not methodology

3.2 BASIC COMPARISON

What are the three Scrum accountabilities, and what does each actually own?

The 2020 Scrum Guide deliberately replaced the word "role" with **accountability** — a subtle but interview-relevant wording change emphasizing what each person is responsible for delivering, not a job title.

ACCOUNTABILITY	OWNS
Developers	Creating a usable Increment each Sprint; the "how" of the technical work
Product Owner	Maximizing product value; owns and orders the Product Backlog; the single point of authority on "what" gets built and in what order
Scrum Master	The Scrum Team's effectiveness; coaching the team, removing impediments, and ensuring Scrum is understood and enacted — not a manager of people or tasks

Developers

Product Owner

Scrum Master

3.3 BASIC WORKFLOW Walk through the Scrum events, in order, for one sprint.

The Sprint itself is a container event that holds all the others — nothing in Scrum happens outside a Sprint.

1. **Sprint Planning** (time-boxed to 8 hours for a 1-month sprint, shorter for shorter sprints) — the team selects Product Backlog items and crafts a Sprint Goal.
2. **Daily Scrum** (15 minutes, every working day) — the Developers inspect progress toward the Sprint Goal and adjust the next day's plan.
3. *(Development work happens continuously through the Sprint.)*
4. **Sprint Review** (up to 4 hours for a 1-month sprint) — the team demonstrates the Increment to stakeholders and gathers feedback that reshapes the Product Backlog.
5. **Sprint Retrospective** (up to 3 hours for a 1-month sprint) — the team inspects how the last Sprint went (people, process, tools) and plans concrete improvements, closing the Sprint.

Time-boxes scale down proportionally for shorter Sprints (e.g. a 2-week Sprint gets roughly half the Planning/Review/Retro time). The Sprint itself is time-boxed to one month or less.

Sprint Planning

Daily Scrum

Sprint Review

Sprint Retrospective

3.4 INTERMEDIATE CONCEPT

What are the three Scrum artifacts, and their associated commitments?

Since the 2020 Scrum Guide, each artifact carries an explicit "commitment" that gives it a measurable target:

ARTIFACT	WHAT IT IS	COMMITMENT
Product Backlog	Ordered, emergent list of everything known to be needed in the product	Product Goal — the long-term objective the backlog serves
Sprint Backlog	Selected Product Backlog items plus the plan to deliver them this Sprint	Sprint Goal — the single objective for this Sprint
Increment	A concrete, working step toward the Product Goal, usable and integrated with all prior increments	Definition of Done — the quality standard every Increment must meet

Product Goal

Sprint Goal

Definition of Done

3.5 INTERMEDIATE COMPARISON

Definition of Ready vs Definition of Done — with example checklists.

Definition of Ready (DoR) is a team's agreed checklist for whether a backlog item is well-formed enough to be pulled into a Sprint — it's a quality gate on the *input* to work, and (unlike DoD) is not an official Scrum Guide artifact, just a widely-adopted team practice. **Definition of Done (DoD)** is the Scrum Guide's official, shared understanding of what "complete" means for any Increment — the quality gate on the *output* of work.

```
// Example Definition of Ready
```

- Acceptance criteria written and agreed with Product Owner
- Story is estimated by the team
- Dependencies identified (external teams, APIs, data)
- UX/design assets attached, if applicable
- Story fits within one Sprint

```
// Example Definition of Done
```

- Code written, peer-reviewed, and merged
- Unit tests written and passing; coverage threshold met
- Integration/regression tests passing in CI
- Documentation updated
- Deployed to staging and verified against acceptance criteria
- Product Owner has reviewed and accepted the Increment

DoR

DoD

quality gate

3.6 INTERMEDIATE CONCEPT

What is a Sprint Goal, and why must it stay fixed even if the backlog items under it change?

The Sprint Goal is the single, coherent objective the Developers commit to for the Sprint — it gives the individual backlog items a shared "why" and lets the team make trade-offs during the Sprint (dropping or swapping a lower-value item) without losing the point of the Sprint entirely. The Scrum Guide is explicit that **scope may be renegotiated between the Developers and Product Owner as more is learned during the Sprint, but the Sprint Goal itself must not be compromised** — it is the one thing genuinely protected once a Sprint starts. This is the mechanism that makes "requirements changed mid-sprint" answerable: the question isn't "can anything change," it's "does the proposed change still serve the Sprint Goal."

Sprint Goal

protected during the Sprint

Mid-sprint, the Product Owner wants to add a new must-have item. What's the correct Scrum response, step by step?

1. **Check it against the Sprint Goal.** If the new item doesn't serve the current Sprint Goal, the default answer is "it waits for the next Sprint" — this isn't obstruction, it's protecting the commitment the whole team made.
2. **If it does serve the Sprint Goal,** the Developers — not the Product Owner unilaterally — assess capacity: can it be absorbed without threatening the Goal, or does something of equivalent size need to come out of the Sprint Backlog in exchange?
3. **Renegotiate scope explicitly** with the Product Owner, in the open, rather than the team silently absorbing extra work (which just produces a burned-out team and a false signal about true velocity).
4. **If it's truly urgent and doesn't fit** even with a trade, that's a signal for a legitimate **Sprint cancellation** — a Scrum Guide-defined, though rare, event the Product Owner alone can invoke when the Sprint Goal has become obsolete.

The answer graders are listening for: this is a negotiation with a clear protocol, not a unilateral "no" or a unilateral "yes."

Sprint cancellation

scope renegotiation

3.8 **ADVANCED** ANTI-PATTERN **What is "Zombie Scrum," and how do you detect it in a retro?**

Zombie Scrum describes a team that performs every Scrum event on schedule — Planning, Daily Scrum, Review, Retro all happen — while producing none of the actual value Scrum is supposed to generate: no real inspection of the Increment happens at Review, retrospective action items are logged but never acted on, and the Daily Scrum degrades into a status report to a manager instead of a peer-to-peer coordination conversation. It looks like Scrum from the outside (the calendar has all the right meetings) while being functionally empty inside. Detecting it in a retro: look for whether the *same* unresolved issues resurface sprint after sprint with no visible attempt at a fix, whether Sprint Review attendance/engagement from real stakeholders is declining, and whether anyone can articulate the current Sprint Goal without looking it up — a team that's forgotten its own Sprint Goal mid-sprint is a strong Zombie Scrum signal.

Zombie Scrum

ceremony without substance

Scrum Master vs traditional Project Manager — why is "PM in Scrum Master clothing" a red flag?

A traditional Project Manager typically owns the plan, assigns tasks to individuals, tracks progress against that plan, and is accountable *for* the team's output to leadership. A Scrum Master owns none of that — they are accountable for the *team's effectiveness at using Scrum*, not for directing the work itself: they coach the team on self-management, facilitate (not run) the events, remove organizational impediments the team can't remove itself, and protect the team from disruptive interference — but they don't assign tasks, don't set the backlog order (that's the Product Owner), and don't manage people (that's typically outside the Scrum Team entirely). "PM in Scrum Master clothing" — someone assigning tasks in standup, chasing individuals for status, making backlog priority calls — is a red flag because it collapses authority back into one person and quietly undoes the self-managing team model Scrum depends on, even while using Scrum's vocabulary.

[servant leadership](#)[coaching vs directing](#)

WHAT THE WEB SOURCES EMPHASIZE

The official Scrum Guide (2020) itself was the clearest authority found — its shift from "roles" to "accountabilities" and its addition of explicit commitments per artifact are recent enough that interviewers use them specifically to separate candidates with current knowledge from those reciting an older Scrum Guide version.

04 Kanban, Scrumban & Other Agile Frameworks

Scrum isn't the only Agile framework, and Atlassian's own Agile Coach material treats Kanban as a first-class alternative, not a lesser cousin. This section covers the frameworks interviewers expect you to be able to place Scrum against.

4.1

BASIC

CONCEPT

What is Kanban, and what are its core practices?

Kanban is a flow-based Agile method (originating from Toyota's manufacturing system) built around three core practices: **visualize the workflow** (a board with columns representing each stage of work, e.g. To Do → In Progress → Review → Done), **limit Work In Progress (WIP)** (cap how many items can sit in each column at once), and **manage flow** (continuously measure and improve how smoothly items move across the board). Unlike Scrum, Kanban has no prescribed roles, no fixed-length iterations, and no required events — work is pulled continuously as capacity frees up, and can be released at any time an item is truly done, not only at a sprint boundary.

visualize workflow

WIP limit

continuous flow

4.2

BASIC

COMPARISON

Scrum vs Kanban — full comparison.

	SCRUM	KANBAN
Cadence	Fixed-length Sprints (1–4 weeks)	Continuous flow, no fixed iterations
Roles	Defined (Developers, PO, Scrum Master)	None prescribed
Core constraint	Sprint Backlog is fixed once the Sprint starts	WIP limit per column, enforced continuously
Change mid-cycle	Discouraged unless it serves the Sprint Goal	Freely pulled in whenever capacity allows
Release cadence	Typically at Sprint boundary	Any time an item clears the board
Best fit	Project-shaped work with a clear backlog to plan against	Support/ops-style work with unpredictable arrival of items

cadence vs flow

4.3 INTERMEDIATE CONCEPT What is a WIP limit, and why does violating it hurt throughput?

A WIP limit caps how many work items may be in a given stage (e.g. "In Progress") at once. It feels counterintuitive — surely doing more work at once gets more done? — but this runs directly against **Little's Law** (average cycle time = average WIP ÷ average throughput): as WIP climbs, either throughput has to climb to match (rarely true — a team's real capacity doesn't scale with how many tickets are open) or cycle time gets worse for every item in flight, because attention is now split and context-switching overhead eats real capacity. A team with 15 tickets "in progress" and a throughput of 3/day isn't 5x more productive than a team with 3 tickets in progress at the same throughput — it's carrying 12 unfinished items' worth of unrealized value and cost, each one aging while nothing on it actually gets finished.

WIP limit

Little's Law

context-switching cost

4.4 INTERMEDIATE CONCEPT What is Scrumban, and when would a team choose it?

Scrumban is a hybrid that layers Kanban's continuous-flow, WIP-limited board on top of some Scrum structure — commonly keeping Sprint Planning-like backlog refinement cadence and retrospectives, while dropping the strict "everything selected at Sprint start stays fixed" constraint in favor of continuous pull as capacity allows. Teams choose it when Scrum's fixed-Sprint-scope commitment doesn't fit their actual work pattern — commonly teams doing a mix of planned feature work and unpredictable, must-interrupt support/production work, where forcing every incoming item to wait for the next Sprint Planning is unrealistic, but pure unstructured Kanban would lose useful planning and retrospective discipline.

Scrumban

hybrid flow + cadence

4.5 INTERMEDIATE CONCEPT

What is Extreme Programming (XP), and which of its practices survive even outside XP teams?

XP is an Agile framework focused specifically on *engineering* discipline — where Scrum defines process structure, XP defines how the code itself should be produced to sustain a fast, safe pace of change. Its core practices: **pair programming** (two developers, one workstation, continuous review), **Test-Driven Development** (tests written before code), **continuous integration** (merging and testing changes frequently, in small batches), **collective code ownership**, and **simple design** (build the simplest thing that could work, refactor as needed later). Notably, TDD and continuous integration have become so widely adopted that most engineering organizations practice them today without identifying as "doing XP" at all — they've become baseline engineering hygiene rather than a distinctive framework choice, which is itself a useful interview observation about how practices outlive their originating framework label.

pair programming TDD continuous integration

4.6 ADVANCED CONCEPT

What is SAFe (Scaled Agile Framework), and what problem does it solve that single-team Scrum doesn't?

Single-team Scrum answers "how does one team of ~5–10 people deliver adaptively." It says almost nothing about coordinating *dozens* of such teams building interdependent pieces of one large product or platform — shared roadmaps, cross-team dependencies, and organization-level budgeting don't disappear just because each team runs its own Sprint. SAFe addresses this by layering additional structure above team-level Scrum/Kanban: a synchronized planning cadence across teams (Program Increment Planning, typically every ~8–12 weeks), cross-team coordination roles (Release Train Engineer), and a defined way for portfolio-level strategy to flow down into team backlogs. The trade-off, and the reason SAFe is controversial in parts of the Agile community, is that this additional structure reintroduces some of the upfront coordination overhead Agile at the team level was designed to minimize — it's a genuine answer to a genuine scaling problem, not a free upgrade.

SAFe Program Increment scaling coordination

Your team's cycle time is increasing even though reported velocity looks stable. What Kanban-style diagnosis would you run?

Velocity (points completed per Sprint) and cycle time (how long an individual item takes from start to finish) can diverge — a team can finish the same total points per Sprint while individual items are quietly taking longer to get through, especially if work is starting earlier and finishing in a last-minute rush ("Sprint boundary batching" rather than steady flow). The Kanban-native diagnostic is a **Cumulative Flow Diagram**: plot the count of items in each workflow stage over time — a widening band in any one stage (commonly "In Review" or "Blocked") reveals exactly where work is piling up, even though the Sprint-level point total looks unchanged. This is the practical case for tracking flow metrics (cycle time, CFD) alongside Scrum's own metrics (velocity) rather than trusting velocity alone as a health signal — a theme that recurs across the estimation section next.

[Cumulative Flow Diagram](#)[cycle time vs velocity](#)

WHAT THE WEB SOURCES EMPHASIZE

Atlassian's Agile Coach material treats Scrum and Kanban as equally valid, workload-dependent choices rather than a maturity ladder — search results specifically warn against treating Kanban as "Scrum without discipline," which is a common but incorrect framing interviewers listen for candidates to avoid.

05 Estimation, Planning & Delivery Metrics

Estimation questions are where interviewers separate candidates who've memorized "story points are relative sizing" from candidates who've actually run a planning session and watched a metric get gamed.

5.1 BASIC CONCEPT What are story points, and why estimate in points instead of hours?

A story point is a unit of **relative effort** — a number expressing how big/complex/risky a story is *compared to other stories*, not a direct time estimate. Teams commonly use a modified Fibonacci scale (1, 2, 3, 5, 8, 13, 20, 40, 100) specifically because the widening gaps at higher values discourage false precision on large, poorly-understood items — you can confidently say "this is roughly twice as complex as that 3-pointer," which is a much easier and more reliable judgment than "this will take exactly 6.5 hours." Estimating in points instead of hours also decouples the estimate from any one individual's speed, so the number stays meaningful as team composition changes.

relative sizing

modified Fibonacci

5.2 BASIC WORKFLOW Walk through Planning Poker end to end.

1. The Product Owner presents the story and its acceptance criteria; estimators ask clarifying questions.
2. Each estimator privately selects a card representing their point estimate — hidden from the others, to avoid anchoring.
3. All cards are revealed simultaneously.
4. If everyone agrees, that value becomes the estimate.
5. If estimates diverge, the highest and lowest estimators explain their reasoning — often surfacing a hidden assumption or risk the rest of the team hadn't considered.
6. The team re-estimates, repeating the reveal-and-discuss cycle until consensus is reached (or a time-box is hit and the team defers to a facilitator's call).

The mechanism's real value isn't the number — it's forcing every disagreement about scope or risk into the open *before* the story is pulled into a Sprint, rather than discovering it mid-development.

Planning Poker

anchoring

hidden-then-reveal

5.3 BASIC CONCEPT What is velocity, and how is it used in sprint planning?

Velocity is the number of story points a team completes (fully meeting the Definition of Done) per Sprint, typically averaged over several recent Sprints to smooth out noise. Once a team has a stable historical velocity, it's used as a planning input — "we've averaged 30 points per Sprint over the last four Sprints, so we should pull roughly 30 points of backlog into the next one" — rather than trying to reason from first principles about capacity every time. It is explicitly a team-specific, backward-looking measure: it says nothing about how "productive" a team is compared to another team, only how much of its own historically-calibrated point scale it tends to clear per Sprint.

velocity

historical average

5.4 INTERMEDIATE COMPARISON

Burndown vs burnup chart — and what does a flat burndown line mid-sprint indicate?

A **burndown chart** plots remaining work (points or hours) against time, trending toward zero by Sprint end — it directly answers "are we on pace to finish." A **burnup chart** plots completed work against time, alongside a separate line for total scope — it directly answers both "how much have we finished" and, crucially, "did scope change mid-Sprint," which a burndown chart alone can obscure (a burndown line that suddenly rises could mean either lost progress or added scope, and can't tell you which). A **flat burndown line mid-sprint** means no story has been marked fully Done in that period — not necessarily that no work happened, since Scrum only counts a story's points once it fully meets the Definition of Done; it's a common early warning sign of stories being too large, too many things in progress at once (the WIP problem from Section 4), or an integration/testing bottleneck near the end of the workflow.

burndown

burnup

all-or-nothing point counting

5.5 INTERMEDIATE CONCEPT What is T-shirt sizing, and when is it preferable to story points?

T-shirt sizing estimates using coarse buckets (XS, S, M, L, XL) instead of numeric points. It's preferable at very early stages — roadmap-level epics, initial backlog grooming — where the item isn't yet broken down enough for the team to reason precisely, and forcing a numeric Fibonacci estimate would create false precision. It's typically converted to story points later, once the epic is broken into real, refined stories that a Planning Poker session can estimate meaningfully.

T-shirt sizing

epic-level estimation

5.6 INTERMEDIATE METRICS

Explain the four DORA metrics, and why they pair throughput with stability.

DORA (Google's DevOps Research and Assessment program) defined four metrics that have become the industry-standard vocabulary for software delivery performance, split across two dimensions:

METRIC	DIMENSION	WHAT IT MEASURES
Deployment Frequency	Throughput	How often code reaches production
Lead Time for Changes	Throughput	Time from a commit being made to it running in production
Change Failure Rate	Stability	% of deployments that cause a production failure requiring a fix
Mean Time to Recovery (MTTR)	Stability	How long it takes to restore service after a failure

The reason they're always reported as a set of four rather than any one alone: optimizing throughput metrics in isolation (ship as fast and often as possible) can quietly wreck stability, and DORA's own research finding — that elite performers achieve *both* high throughput and high stability simultaneously — is exactly what makes the pairing meaningful evidence rather than a philosophical claim.

Deployment Frequency Lead Time Change Failure Rate MTTR

5.7 ADVANCED SCENARIO

Management asks you to use velocity to compare two different teams' productivity. What's wrong with this, and what would you say instead?

Story points are deliberately *not* calibrated across teams — a "5" on Team A's scale and a "5" on Team B's scale are independently, locally meaningful and carry no shared unit of comparison; one team's 5-pointer might be another team's 2 or 13, depending entirely on how each team's own Planning Poker sessions have calibrated their scale over time. Comparing raw velocity across teams therefore compares nothing real, and worse, creates a strong incentive for teams to inflate their own point values over time to "look" more productive — a well-documented Goodhart's Law failure mode (once a measure becomes a target, it stops being a good measure). The better response: if leadership genuinely needs a cross-team comparison, point them toward metrics that *are* comparable across teams by construction — cycle time, throughput (item count, not points), or the DORA metrics — none of which depend on a team-local, self-calibrated scale.

Goodhart's Law points aren't comparable across teams

Why is "estimate once, lock it in" considered an anti-pattern given the Cone of Uncertainty?

Per Section 1's Cone of Uncertainty, an estimate made at the point of least information (project kickoff) is mathematically guaranteed to have the widest error bars of any point in the project — yet "estimate once, lock it in" treats that earliest, least-informed number as a permanent commitment, then judges the team against it for the rest of the project regardless of what's since been learned. This produces two predictable failure modes: either the team pads early estimates heavily to protect against the (correctly anticipated) unknowns, defeating the purpose of estimating honestly, or the team hits real, newly-discovered complexity mid-project and is forced to choose between quietly cutting quality/scope or visibly "failing" against a number that was never realistic to begin with. The Agile alternative — re-forecasting from current, better-informed velocity/cycle-time data at regular intervals — isn't sloppier planning, it's planning that acknowledges where on the cone you actually are.

re-forecasting

estimate padding

Cycle time vs lead time — how do they diagnose different bottlenecks?

Lead time measures from the moment work is *requested* (e.g. added to the backlog) to the moment it's delivered — it captures the full customer-experienced wait, including time spent sitting unprioritized in the backlog. **Cycle time** measures only from the moment work actually *starts* to the moment it's delivered — it isolates the team's execution speed, stripped of queueing delay. A widening gap between the two (lead time growing while cycle time stays flat) points to a **prioritization/backlog-grooming problem** — items are waiting too long before anyone starts them. A cycle time that's growing on its own points to an **execution problem** — too much WIP, unclear requirements, or a bottleneck in the workflow itself (as in Section 4.7's Cumulative Flow diagnosis). Conflating the two metrics is a common mistake that leads teams to "fix" the wrong stage of the pipeline.

lead time

cycle time

queueing vs execution delay

WHAT THE WEB SOURCES EMPHASIZE

Mountain Goat Software's estimation material and multiple Glassdoor reports both foreground "how did you estimate stories in your last team" as a genuinely asked question, and GitLab/DORA's own published research is explicit that throughput and stability metrics must be read together, not separately — both findings shaped how this section is weighted toward pairing metrics rather than presenting any single number as sufficient.

06 Requirements, User Stories & Change

Where SDLC theory meets the day-to-day artifact every Agile team actually writes: the user story. This section also covers the single most commonly reported real scenario — a Product Owner changing their mind mid-sprint.

6.1 **BASIC** CONCEPT What is a user story, and what's its standard template?

A user story is a short, plain-language description of a feature written from the perspective of the person who wants it, deliberately kept small enough to fit in a single Sprint and deliberately light on implementation detail — the "how" is left for the team to work out, and detailed conversation with stakeholders is expected to fill in what the written card alone can't capture.

```
As a [type of user]
I want [an action or capability]
so that [the benefit or value it delivers]

// Example
As a returning customer
I want my saved payment method to appear at checkout
so that I can complete a purchase without re-entering card details
```

The value clause ("so that...") is the part most often skipped by inexperienced teams, but it's what lets the Product Owner and Developers judge trade-offs sensibly during the Sprint — without a stated "why," any proposed simplification looks equally acceptable or unacceptable.

user story template

value clause

What are acceptance criteria, and how does Given/When/Then (Gherkin) make them testable?

Acceptance criteria are the specific, checkable conditions a story must satisfy to be considered Done — they turn a story's intent into something that can be objectively verified, rather than left to interpretation. The **Gherkin** format (Given/When/Then), originating from Behavior-Driven Development, structures each criterion as a precondition, an action, and an expected outcome — which has the useful side effect of being directly translatable into an automated acceptance test.

```
Given a returning customer with a saved payment method
When they reach the checkout page
Then the saved payment method is pre-selected
And they can complete checkout without re-entering card details

Given a returning customer whose saved card has expired
When they reach the checkout page
Then they are prompted to enter a new payment method
```

acceptance criteria

Gherkin

Given/When/Then

6.3 BASIC CONCEPT What does INVEST mean for a well-formed user story?

- **Independent** — can be developed and delivered without hard sequencing on another story
- **Negotiable** — a starting point for conversation, not a rigid contract fixed in every detail
- **Valuable** — delivers something a real user or stakeholder actually cares about
- **Estimable** — the team has enough clarity to size it
- **Small** — fits comfortably within a single Sprint
- **Testable** — has clear, verifiable acceptance criteria

INVEST functions as a diagnostic checklist during backlog refinement: a story failing "Independent" or "Small" is the most common reason a story silently blows through a Sprint without finishing.

INVEST

6.4 INTERMEDIATE COMPARISON

What is scope creep, and how does it differ from a legitimate, well-managed requirement change?

Scope creep is **uncontrolled, unacknowledged** growth in what a project or Sprint is expected to deliver — small additions accepted informally, without re-evaluating schedule, cost, or what else might need to be dropped in exchange. A legitimate requirement change is the same underlying event — new information causing a change in what's needed — but handled through an explicit process: the trade-off is surfaced, agreed to by the people accountable for it (Product Owner and Developers, per 3.7), and something is consciously renegotiated in exchange, rather than simply absorbed. The distinguishing test isn't "did the requirements change" — in Agile, they're expected to — it's "was the change explicit and its cost acknowledged, or did it just quietly happen."

scope creep

explicit trade-off

6.5 INTERMEDIATE COMPARISON

Epics vs Features vs User Stories vs Tasks — how do they nest?

LEVEL	SCOPE	TYPICAL SIZE
Epic	A large body of work spanning multiple releases or quarters	Months
Feature	A significant, shippable capability within an Epic	Weeks, possibly multiple Sprints
User Story	A single, Sprint-sized piece of user-facing value	Fits in one Sprint
Task	A technical, non-user-facing unit of work needed to complete a story	Hours to a couple of days

Backlog refinement is largely the ongoing work of breaking Epics down into Features and Features into Stories small enough to satisfy INVEST — a story that's really an unbroken Epic is the most common reason a Sprint Backlog item never finishes.

Epic

Feature

Story

Task

The Product Owner keeps changing acceptance criteria mid-sprint, after work has started. What's the right conversation?

Start from what's actually protected: per 3.6, the Scrum Guide protects the Sprint Goal, not the literal wording of every acceptance criterion — so the first move is diagnosing which case this is. If the changes are refinements that still serve the same Sprint Goal (the PO is clarifying, not redirecting), that's normal and expected collaboration — Scrum explicitly wants the PO engaged throughout the Sprint, not just at the boundaries. If the changes are repeatedly invalidating work the Developers have already built, that's a signal the story wasn't actually Ready when it was pulled in (a Definition of Ready failure, 3.5) or that the PO is using the Sprint as a discovery process instead of using backlog refinement and the Sprint Review for that purpose. The conversation to have — ideally raised in the next Daily Scrum or, if it's a pattern, the Retrospective — names the cost concretely ("this is the third time this Sprint we've rebuilt this feature, and it's putting the Sprint Goal at risk") rather than either silently absorbing it or refusing further PO input.

DoR failure vs legitimate refinement

What questions should a team ask before accepting a change request mid-sprint?

1. **Does it serve the current Sprint Goal?** If not, it's next-Sprint work by default.
2. **What's the size, roughly?** Even a rough estimate is needed before agreeing to anything.
3. **What comes out in exchange?** Sprint capacity doesn't expand just because a new priority appeared.
4. **Is it genuinely urgent, or does it just feel urgent to whoever raised it?** Distinguishing "must happen this Sprint" from "would be nice to have sooner" is most of the actual negotiation.
5. **Who has the authority to say yes?** This should be a Developers + Product Owner conversation, not a decision made unilaterally by either side or imposed by an outside stakeholder.

change-request checklist

What is requirements volatility, and how does a team measure and manage it?

Requirements volatility is the rate at which requirements change after they've been captured — commonly tracked as the number or percentage of backlog items that get materially re-scoped, re-prioritized, or removed after initial refinement, measured per release or per quarter. Some volatility is healthy and expected — it's the whole point of iterative development responding to real feedback — but a consistently high rate, especially concentrated in items that had already entered a Sprint, is a leading indicator of an underlying problem: an under-engaged Product Owner not validating direction with stakeholders early enough, or a backlog refinement cadence too infrequent to catch changing priorities before they become disruptive mid-Sprint. Teams manage it by tightening the Definition of Ready (catching ambiguity before a story is pulled in, not after), increasing refinement frequency, and tracking volatility explicitly as a Retrospective input rather than treating each instance as an isolated, unrelated surprise.

[requirements volatility](#)[backlog refinement cadence](#)

WHAT THE WEB SOURCES EMPHASIZE

iZenBridge and ProjectManagement.com both frame the "accept changes within a Sprint" question the same way independently: the deciding factor search sources converge on is whether the change serves the Sprint Goal, not a blanket yes/no policy — this shaped 6.6 and 6.7 directly.

07 Software Testing & Quality Assurance

SE interviews increasingly blend process knowledge with testing strategy — expect at least one question connecting "how does your team develop" to "how does your team know it works."

7.1

BASIC

CONCEPT

Explain the Test Pyramid — its layers, and why the base is the biggest.

The Test Pyramid is a strategy metaphor for how a healthy automated test suite should be shaped: a large base of **unit tests** (fast, isolated, testing one small piece of logic in milliseconds), a smaller middle layer of **integration tests** (verifying multiple components work together correctly — a service talking to a real database, say), and a very small top layer of **end-to-end/UI tests** (exercising the whole system the way a real user would). The pyramid shape reflects a direct cost/speed trade-off: unit tests are cheap to write, run in milliseconds, and pinpoint failures precisely, so a suite should have thousands of them; end-to-end tests are slow, brittle (a UI change can break tests unrelated to what actually regressed), and expensive to maintain, so a healthy suite has relatively few, reserved for the critical user journeys nothing else can verify.

Test Pyramid

unit tests

integration tests

end-to-end tests

7.2

BASIC

COMPARISON

Unit vs Integration vs End-to-End vs Acceptance testing.

LEVEL	TESTS	SPEED	OWNED BY
Unit	A single function/class in isolation, dependencies mocked	Milliseconds	Developers
Integration	Multiple real components together (e.g. service + real DB)	Seconds	Developers
End-to-End (System)	The whole application, as a user would experience it	Minutes	Developers / QA
Acceptance	Does the system meet the business/user's actual requirement	Varies	QA / Product Owner

test levels

7.3

BASIC

WORKFLOW

What is TDD, and what's the Red-Green-Refactor cycle?

1. **Red** — write a test for the behavior you're about to build. It fails, because the behavior doesn't exist yet.
2. **Green** — write the minimum code needed to make that test pass. Not the "best" code yet — just enough to pass.
3. **Refactor** — now that the behavior is locked in by a passing test, clean up the implementation (remove duplication, improve naming) without changing behavior, re-running the test after each change to confirm it still passes.

Writing the test first, rather than after, forces a concrete definition of "done" for the smallest possible unit of behavior before any implementation exists, and the accumulated test suite becomes a safety net that makes later refactoring safe rather than terrifying.

Red-Green-Refactor

TDD

7.4

INTERMEDIATE

CONCEPT

What is regression testing, and why does it matter more as a codebase grows?

Regression testing re-runs existing tests after a change to confirm that previously-working functionality hasn't broken — "regressed" — as a side effect of new work. Its importance scales with codebase size and age because the surface area of things a change could unintentionally affect grows continuously, while any individual engineer's mental model of the whole system shrinks relative to that surface area — a change to a shared utility function that looks locally safe can break a dozen unrelated call sites nobody making the change was even aware of. This is exactly why an automated regression suite, not manual re-testing, is the only approach that scales: it's the mechanism that lets a team keep shipping quickly without re-verifying the entire system by hand on every change.

regression testing

automated safety net

Verification vs validation — "building it right" vs "building the right thing."

Verification asks: does the system conform to its specification? ("Are we building it right?") — this is what unit and integration tests primarily check, against a design or requirements document.

Validation asks: does the system actually meet the real user's need? ("Are we building the right thing?") — this is what acceptance testing, Sprint Review demos, and real usage feedback check, and it's possible to pass every verification test while still failing validation, if the specification itself was wrong. This distinction is the academic backbone of why Agile insists on frequent Sprint Reviews with real stakeholders rather than trusting a passing test suite alone as proof of success — a test suite can only verify against what was written down, never validate that what was written down was the right thing to build.

verification

validation

7.6 INTERMEDIATE ANTI-PATTERN What is the "Ice Cream Cone" anti-pattern in test strategy?

The Ice Cream Cone is the Test Pyramid inverted: a large number of slow, brittle, expensive end-to-end/manual UI tests at the top, and relatively few fast unit tests at the bottom. It typically emerges organically — teams under deadline pressure reach for "just test it through the UI, it covers everything" as the path of least resistance, rather than investing in the harder upfront work of designing code to be unit-testable in isolation. The consequence compounds over time: the test suite becomes slow (minutes or hours to run, discouraging frequent runs), flaky (UI tests fail for reasons unrelated to real regressions — timing, environment), and expensive to maintain (a small UI change breaks dozens of tests), which paradoxically makes the team trust and use their own test suite less, not more, exactly the opposite of what a test suite is for.

Ice Cream Cone anti-pattern

flaky tests

A feature passes all automated tests but breaks in production for an untested use case. What's the retrospective action?

The immediate fix (add a test covering the missed case, patch the bug) is necessary but not the interesting part of the answer — that's just closing one instance. The retrospective-worthy question is *why* that use case wasn't covered: was it missing from the acceptance criteria entirely (a requirements-gathering gap, pointing back to Section 6), was it known but deprioritized under time pressure (a Definition of Done that isn't actually being enforced), or is it a genuinely novel edge case nobody could reasonably have anticipated (in which case the real fix is a better production monitoring/alerting signal, not more pre-release testing)? Treating every production bug as "we need more tests" without this diagnosis leads to bloated, low-signal test suites that don't actually target where real gaps are; treating it as purely bad luck skips a genuine, fixable process gap. The mature answer distinguishes which of the three it actually was.

[root-cause categorization](#)[edge-case coverage](#)

What is shift-left testing, and how does it change the economics of defect cost?

"Shift-left" means moving testing activities earlier in the SDLC timeline — writing tests alongside or before code (TDD), running static analysis and linting at commit time, and involving QA in requirements/design review rather than only after implementation is "done." The economic argument (well-established in software engineering research going back decades) is that the cost of fixing a defect grows roughly exponentially the later it's discovered — a requirements misunderstanding caught in a design review costs a conversation; the same misunderstanding caught after release costs a production incident, a hotfix, and possibly damaged customer trust. Shift-left is the direct practical implementation of that cost curve: front-load the cheap-to-fix discovery of problems, rather than relying on the traditional, expensive-to-fix "test at the end" model that both Waterfall and the naive reading of "move fast" both tend to produce.

[shift-left](#)[cost-of-defect curve](#)

WHAT THE WEB SOURCES EMPHASIZE

Katalon, TestRail, and Codecademy's testing material converge on the same pyramid shape and the same warning against the inverted "Ice Cream Cone" — the consistency across independent commercial and educational sources is itself evidence this is treated as settled industry consensus, not a stylistic preference.

08 CI/CD, DevOps & Release Management

The layer where "we finished the Sprint" becomes "it's actually running for users."
Increasingly tested even in non-DevOps SE interviews, since nearly every engineer now touches a pipeline.

8.1

BASIC

COMPARISON

Continuous Integration vs Continuous Delivery vs Continuous Deployment — where exactly does each stop?

PRACTICE	GUARANTEES	STOPS AT
Continuous Integration (CI)	Code is merged frequently and validated by an automated build/test on every merge	A green build — no deployment implied
Continuous Delivery (CD)	Every change that passes CI is kept in a genuinely deployable state	A human decides when to actually release it
Continuous Deployment	Every change that passes the pipeline is automatically released	Nothing — release is fully automated

The most commonly confused pair in interviews is the last two: Continuous *Delivery* means "we could release right now, any time, with one click" — Continuous *Deployment* means "we do release, automatically, every time."

CI

Continuous Delivery

Continuous Deployment

8.2

BASIC

WORKFLOW

What are the typical stages of a CI/CD pipeline?

1. Source

→ commit/PR triggers the pipeline
2. Build

→ compile / package the application
3. Test

→ unit → integration → (sometimes) e2e, run automatically
4. Artifact

→ store the versioned, tested build output
5. Staging

→ deploy to a production-like environment for final checks
6. Deploy

→ release to production (manual gate, or automatic)
7. Monitor

→ watch real production signals post-release
8. Rollback

→ automated or manual reversion if monitoring detects a problem

A pipeline that stops at step 3 (build + test, no deployment automation) is CI only; one that runs through step 6 without a human gate is Continuous Deployment.

pipeline stages

8.3

INTERMEDIATE

COMPARISON

Blue-Green vs Canary vs Rolling deployment — risk and rollback trade-offs.

STRATEGY	HOW IT WORKS	ROLLBACK SPEED	RISK EXPOSURE
Blue-Green	Two full identical environments; traffic is switched entirely from old (blue) to new (green) at once	Instant — switch traffic back	All-or-nothing; a bad release affects 100% of users the moment it's switched
Canary	New version rolled out to a small % of traffic first, then gradually increased if healthy	Fast — halt the rollout, route traffic away from canary	Limited — only the canary slice is exposed initially
Rolling	Instances updated gradually, one batch at a time, old and new versions coexist mid-rollout	Slower — must roll back batch by batch	Moderate; both versions serve traffic simultaneously during rollout

Blue-Green

Canary

Rolling

8.4 INTERMEDIATE CONCEPT

What is a feature flag, and how does it decouple deployment from release?

A feature flag is a runtime conditional that gates whether a piece of code is active, controlled by configuration rather than by which code is deployed. It separates two ideas that are easy to conflate: **deployment** (the code is running in production) and **release** (real users can actually reach and use the feature). With flags, a team can deploy incomplete or risky code to production continuously — dark, switched off — and separately decide, independently of any deploy, exactly when and to whom to turn it on (internal testers first, then a percentage rollout, then everyone). This is what makes Continuous Deployment survivable for genuinely risky changes: the deploy itself becomes low-stakes, because the flag — not the deploy — is the actual release decision, and turning a flag back off is a much faster mitigation than a code rollback.

feature flag

deployment ≠ release

8.5 INTERMEDIATE CONCEPT

What does "you build it, you run it" mean, and how does it change accountability?

Coined at Amazon and now widespread DevOps culture, "you build it, you run it" means the team that writes a service is also the team on-call for it in production — there's no separate Ops team that inherits an unfamiliar codebase during an outage. This changes accountability sharply: developers now directly feel the operational consequences of a poorly-tested change or a sloppy rollback plan (a 3am page), which creates a much tighter, faster feedback loop pushing quality and observability concerns earlier into development, compared to the classic Dev/Ops split where a change gets thrown "over the wall" and the people who wrote it may never learn how badly it actually behaved in production. The trade-off is real too: it demands developers acquire genuine operational skill (monitoring, incident response), not just coding skill.

you build it, you run it

on-call ownership

A canary deployment shows a small error-rate increase.

What's your decision framework — rollback or proceed?

1. **Is the increase statistically meaningful**, or within normal noise for that traffic volume? A tiny canary slice can show a misleadingly large percentage swing from very few actual errors.
2. **Is the error correlated with the new version**, or is it an unrelated, pre-existing issue that happened to coincide with the rollout window?
3. **What's the blast radius if it's real** — is this error type user-visible and severe (failed checkouts) or low-severity (a logging warning)?
4. **What's the cost of waiting to gather more signal** versus rolling back now — is the canary slice small enough that a few more minutes of data is cheap, or is real user harm accumulating right now?

The default bias, absent a clear "this is noise" signal, should favor rollback — a canary's entire purpose is cheap reversibility, and the cost of a false-positive rollback (re-running the pipeline) is almost always far lower than the cost of a false-negative "proceed" decision that ships a real regression to 100% of users.

canary decision framework

blast radius

Rollback vs roll-forward — when is each the right incident response?

A **rollback** reverts to the last known-good deployed version — fast, and doesn't require understanding the root cause first, which makes it the right default when user impact is severe and ongoing. A **roll-forward** fix ships a new, corrective change on top of the current (broken) version instead of reverting — appropriate when reverting isn't actually safe or possible (e.g. the bad deploy included a database migration that already ran and isn't cleanly reversible, so going back to old code against the new schema would break differently) or when the fix is already known, trivial, and faster to ship than a full rollback-and-redeploy cycle would be. The general production-engineering guidance found across sources: default to rollback for speed and safety unless a specific, identified reason (usually data/schema irreversibility) makes roll-forward the only real option.

rollback

roll-forward

irreversible migrations

How do the four DORA metrics reveal whether CI/CD investment is actually paying off?

A team can invest heavily in pipeline tooling and still not actually be delivering better — the DORA metrics are the check on whether that investment shows up in outcomes rather than just activity. Rising **Deployment Frequency** and falling **Lead Time** show the pipeline is genuinely reducing friction between "code written" and "code running," not just that more meetings about CI/CD are happening. Critically, those two alone can be gamed or coincidentally rise while quality quietly erodes — which is exactly why they must be read alongside **Change Failure Rate** and **MTTR**: if deploys are getting faster and more frequent while change failure rate also climbs, the "improvement" is actually just accumulating risk faster, not reducing it. A real, healthy CI/CD investment moves all four in the right direction together — faster and more frequent, without deployments becoming riskier or recovery becoming slower — which is precisely the pattern DORA's research found separates elite-performing teams from the rest.

DORA as an outcome check

throughput without stability is risk

WHAT THE WEB SOURCES EMPHASIZE

CI/CD interview-question aggregators (MentorCruise, Katalon, InterviewBit) consistently rank "CI vs Delivery vs Deployment" and "blue-green vs canary" as the top-tested distinctions, and specifically flag rollback-strategy scenario questions as increasingly common in 2026-era interview reports — both weighted heavily into this section.

09 Agile Anti-Patterns & Technical Debt

Senior and Scrum Master interviews lean heavily on this category — anyone can recite what Scrum *should* look like; this section tests whether you can recognize it quietly failing.

What is a "ScrumBut," and why is it a red flag rather than harmless customization?

A ScrumBut is a team practice described in the pattern "we do Scrum, but [we skip the Retrospective / the Scrum Master also assigns tasks / we don't really have a Product Owner]" — a deviation from the framework, typically adopted to avoid a difficulty Scrum surfaced, rather than to fix the underlying problem it surfaced. The reason it's treated as a warning sign rather than healthy tailoring: Scrum's events and artifacts are each there to surface a specific class of problem (a Retrospective surfaces process dysfunction; a real Product Owner surfaces priority conflicts) — dropping the practice doesn't remove the underlying problem, it just removes the team's visibility into it. Some deviation from textbook Scrum is normal and even encouraged (Scrum explicitly says use only what you need); the distinguishing question a ScrumBut fails is whether the team consciously improved on Scrum's purpose, or just quietly dropped the part that was uncomfortable.

ScrumBut

process debt

What is a "Hardening Sprint," and why is it considered an anti-pattern?

A Hardening Sprint is a Sprint set aside specifically for bug-fixing, stabilization, and cleanup, separate from Sprints doing "real" feature work. It's an anti-pattern because it implicitly admits that the Definition of Done used in regular Sprints isn't actually rigorous enough — if an Increment genuinely met a solid DoD each Sprint (tested, integrated, reviewed), there would be no growing backlog of instability requiring a dedicated Sprint to address later. Practiced repeatedly, it also delays quality feedback exactly when Section 7.8's shift-left argument says it should be fastest — defects accumulate for Sprints before the hardening pass surfaces them, by which point they're more expensive to fix and harder to trace to their origin. The fix isn't "add hardening Sprints," it's tightening the Definition of Done so quality debt doesn't need a separate cleanup Sprint to begin with.

Hardening Sprint

DoD isn't rigorous enough

What is technical debt, and what's the difference between deliberate and accidental debt?

Ward Cunningham's original metaphor: choosing a quick, imperfect implementation to ship faster is like taking on financial debt — it buys speed now, at the cost of ongoing "interest" (the accumulated extra effort every future change to that code will require until the debt is paid down by refactoring).

Deliberate debt is taken on consciously — a team explicitly decides "we'll hardcode this for the demo and refactor properly next Sprint," with the trade-off named and, ideally, tracked. **Accidental (or "inadvertent") debt** accumulates without anyone deciding to take it on — from evolving requirements the original design didn't anticipate, from a team's own skill/knowledge gaps at the time, or simply from normal software aging as the surrounding system changes around code that hasn't. The interview-relevant distinction: deliberate debt, tracked and repaid on schedule, is a legitimate engineering trade-off; debt that's neither tracked nor ever repaid — regardless of which kind it started as — is what actually causes the failure modes in 9.2 and 9.4.

Ward Cunningham

deliberate vs accidental debt

What does constant context-switching from too much WIP do to a team's real throughput?

Directly connecting back to Section 4.3's WIP-limit argument: a team juggling many simultaneously "in progress" items looks busy — every person has something active — but real completed throughput can be far lower than it appears, because each context switch between unrelated tasks carries a real cognitive cost (re-loading context, re-establishing where you left off) that doesn't show up on any board as "work," yet consumes real capacity. The visible symptom search sources repeatedly describe: developers jumping fire to fire, nothing quite finishing, technical debt never getting paid down because there's never a moment without an "urgent" interruption competing for attention — the team isn't lazy or incapable, it's structurally prevented from finishing anything by its own WIP levels.

context-switching cost

looks busy, isn't finishing

Retros keep surfacing the same complaint sprint after sprint, with no change. What's actually broken?

The Retrospective itself isn't broken — it's doing its job, surfacing the real issue repeatedly and honestly. What's broken is the step *after* the Retrospective: action items aren't being tracked, owned, or followed up on, so raising the same issue becomes a ritual with no consequence, which is also precisely the "Zombie Scrum" pattern from 3.8. The fix isn't a better Retrospective format — it's treating retro action items with the same rigor as backlog items: assign an owner, give it a visible place on the board, and — critically — open the *next* Retrospective by reviewing whether last time's action items actually happened, before generating new ones. A Retrospective that never checks its own follow-through is optimized for venting, not for the "adapt" half of "inspect and adapt."

[action-item follow-through](#)[inspect and adapt](#)

9.6 **ADVANCED** ANTI-PATTERN **What is "Watermelon status reporting," and what causes it?**

Watermelon status reporting describes a project or Sprint that reports "green" (on track) at every status checkpoint — right up until it's revealed to actually be "red" (in serious trouble) underneath, green skin over a red interior. It's caused by a mix of incentive and psychological-safety failures: individuals or teams under pressure to report good news (fear of blame, a culture that punishes bad news rather than the underlying problem), status reporting granularity too coarse to show partial risk (a single red/yellow/green flag can't express "80% fine, one serious blocker"), and — connecting to 3.8 and 9.5 — Retrospectives or Daily Scrums that aren't functioning as genuine, safe spaces to surface a problem before it's unrecoverable. The structural fix is the same across all of these: status needs enough granularity to show partial risk honestly, and the team culture needs to reward surfacing a problem early over appearing fine until it can't be hidden any longer.

[Watermelon reporting](#)[psychological safety](#)

Velocity is climbing every sprint, but stakeholders are unhappier with output quality. What's going on, and how do you counter it?

This is Goodhart's Law in action, the same failure mode named in 5.7, but showing up within a single team rather than across teams: once velocity becomes a number people are implicitly judged on, teams find ways to make the number go up that don't require actually delivering more real value — inflating point estimates over time (the same work gets called "8" this quarter that was called "5" last quarter), quietly narrowing acceptance criteria to make stories easier to mark Done, or deferring the unglamorous-but-necessary work (tests, refactoring, edge cases) that doesn't show up as points but does show up later as stakeholder-visible quality problems. The counter isn't to abandon velocity — it's to stop treating it as a standalone success metric and pair it, exactly as Section 5 argued for DORA, with independent signals that can't be gamed the same way: real Change Failure Rate, actual bug counts post-release, and direct stakeholder feedback at Sprint Review — a rising velocity that isn't accompanied by stable or improving quality signals elsewhere is itself the diagnostic.

[Goodhart's Law](#)[metric gaming](#)[pairing metrics to prevent gaming](#)

WHAT THE WEB SOURCES EMPHASIZE

Scrum.org's own published anti-pattern taxonomy and Parabol's aggregated list both converge on the same three root causes across dozens of named anti-patterns: unresolved conflict avoidance, an unenforced Definition of Done, and Product Owner disengagement — which is why this section repeatedly traces surface-level symptoms (Hardening Sprints, Watermelon reporting, stalled retros) back to those same few structural causes rather than treating each as an isolated failure.

Sources consulted

Compiled from current practitioner and authoritative sources as of September 2026. Where a source says "frequently asked" or "commonly reported," that phrasing reflects the aggregated interview-report language found in the cited source itself, not an invented popularity claim.

[The 2020 Scrum Guide \(official, scrumguides.org\)](#)[Scrum.org — Scrum Guide 2020 Updates](#)[Atlassian Agile Coach — Agile vs Scrum](#)[Atlassian — Scrumban](#)[GeeksforGeeks — SDLC Interview Questions](#)[GeeksforGeeks — Complete Guide to SDLC Models](#)[Glassdoor — Agile Interview Questions \(aggregated reports\)](#)[Scrum Institute — 60 Top Scrum Master Interview Q&A](#)

[Mountain Goat Software — Planning Poker](#)

[DORA — Software Delivery Performance Metrics](#)

[GitLab — DORA Metrics Guide](#)

[iZenBridge — Accepting Requirement Changes Within a Sprint](#)

[Scrum.org — Scrum Team Failure: Anti-Patterns Taxonomy](#)

[Parabol — 66 Scrum Anti-Patterns](#)

[Katalon — Software Testing Interview Questions](#)

[TestRail — The Testing Pyramid](#)

[MentorCruise — 80 CI/CD Interview Questions](#)

[BugBug — Mastering CI/CD: Interview Questions and Answers](#)

Delivery Interview Manual · Software Engineering / SDLC / Agile / Scrum Edition · Use the "Save as PDF" button (top of page) to export via your browser's print dialog for offline study.